

Reviewer Pack — Minimal L^2 -Norm of Fourier Coefficients for AC0 Parity

Entry 02d965bdf405 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 00:20:54 UTC

Minimal L^2 -Norm of Fourier Coefficients for AC0 Parity

Entry ID: 02d965bdf405 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 00:20:54 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis over the Complex Plane **Field B** (complexity object): AC⁰ Circuit Complexity

Statement:

[‘For any AC⁰ circuit C computing the PARITY function on n inputs, the L^2 -norm of its Fourier coefficients is lower bounded by a constant c times $\log(\text{size}(C))$. Specifically, for all circuits C with size $n \leq 40$, $\|F(C)\|_2 \geq c \cdot \log(n)$.’, ‘Equivalently, there exists a function $f: [1, n] \rightarrow \mathbb{R}$ such that for any AC⁰ circuit C computing the PARITY function on n inputs, $\|F(C) - f\|_2 \leq \epsilon$.’, ‘No counterexample with $n \leq 40$ can have a smaller L^2 -norm of its Fourier coefficients than the smallest possible value for circuits of that size computing PARITY.’]

Rationale (proposer’s reasoning):

[‘Fourier analysis has been successfully used to study circuit complexity, particularly in proving lower bounds. However, direct application to AC⁰ circuits is limited due to their inherent structure. By studying the L^2 -norm of Fourier coefficients, we aim to find a new invariant that captures essential properties of AC⁰ circuits computing PARITY.’, ‘This approach may expose deeper connections between the geometric properties of functions and circuit complexity, providing insights into the nature of computation at the AC⁰ level.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ebc170c5ec38b4f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an AC0 circuit computing PARITY on n inputs, if the L^2 -norm of its Fourier coefficients is $\|F(C)\|_2 \geq c \cdot \log(n)$

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	UNCERTAIN	0.95	HITS	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def determinant(A):
    n = len(A)
    det = Fraction(1)
    for i in range(n):
        if A[i][i] == 0:
            return 0
        det *= A[i][i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
    return det

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[Fraction(0) for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
```

```

        for k in range(p):
            C[i][k] += A[i][j] * B[j][k]
    return C

def fourier_coefficients(circuit, n):
    F = [Fraction(0) for _ in range(2**n)]
    for i in range(2**n):
        sum_val = Fraction(0)
        for k in range(n):
            sum_val += circuit[i] * math.exp(-2j * math.pi * k * i / (2**n))
        F[i] = sum_val
    return F

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    circuit = [random.choice([-1, 1]) for _ in range(2**n)]

    F = fourier_coefficients(circuit, n)
    norm_F = sum(x * x.conjugate() for x in F).real

    if norm_F == 0:
        return {
            "metric_name": "L^2-norm of Fourier coefficients",
            "metric_value": norm_F,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "circuit_all_zeros"
        }

    c = Fraction(1, n)
    if norm_F >= c * math.log(n):
        return {
            "metric_name": "L^2-norm of Fourier coefficients",
            "metric_value": norm_F,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }
    else:
        return {
            "metric_name": "L^2-norm of Fourier coefficients",
            "metric_value": norm_F,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"norm_F < c * log(n) with n={n}, norm_F={norm_F}, c*log(n)={c*math.log(n)}"
        }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

```

```

results.append(trial_result)

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"norm_F < c * log(n)\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_416d62ab.py", line 101, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_416d62ab.py", line 65, in run_trial
    F = fourier_coefficients(circuit, n)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_416d62ab.py", line 56, in fourier_coefficients
    sum_val += circuit[i] * math.exp(-2j * math.pi * k * i / (2**n))
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: must be real number, not complex

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15651
2	preregistration	ollama_remote	glm4:latest	0	0	10794
3	novelty	ollama_remote	glm4:latest	0	0	11518
4	novelty	ollama_remote	glm4:latest	0	0	8952
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12317
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9027
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7441
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12061
9	judge	ollama_remote	glm4:latest	0	0	11854

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99615 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/02d965bdf405.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/02d965bdf405.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/02d965bdf405.tar.gz` (if generated)